

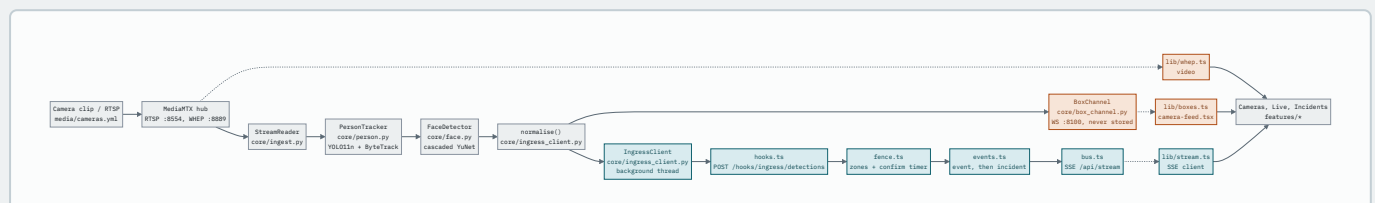
Human Tracking & Face Detection

How a person walking past a camera becomes a tracked box on an operator's screen and, if it matters, a **hash-chained incident** at the post — traced through every file that touches it, on both sides of the wire.

This is one of two domains sharing the `ibvap/` folder. People (detection + tracking + faces — this document) get `people_run.py` and `people_service.py`. Vehicles and number-plate OCR get `run.py` and `service.py`. The split exists because it was broken once: the vehicle work originally replaced the detector inside the shared `run.py/service.py` files in place, which silently deleted human tracking — this platform's first scope item — to make room for its fourth. Two pairs of files now, so neither domain can overwrite the other by editing “the” entry point again.

01 The whole path, one picture

Four processes. **HOT** is the live overlay a browser tile draws from — ephemeral, never stored. **COLD** is the record the backend keeps — durable, hash-chained, the only thing that survives a restart.



Nothing here knows what a zone is. `people_service.py` pulls video and produces boxes; whether a box crossing a line is worth an alarm is decided one layer up, in `fence.ts`. Keeping the vision service ignorant of that is what makes “one program, many configuration files” true rather than aspirational.

02 Frame to incident, in order

Same diagram, narrated. This is the path one detector call actually takes.

-
- 01 **The hub holds one copy.** MediaMTX pulls the clip or camera once and serves it two ways — RTSP to whichever machine runs detection, WHEP to every browser tab. Cheap IP cameras cap at 2–4 concurrent sessions; nothing downstream ever touches the camera directly. `media/mediamtx.yml`, generated by `media/configure.py`
-
- 02 **The reader keeps only the newest frame.** `cv2.VideoCapture.read()` pulls sequentially, so if the detector falls behind, an un-managed buffer goes stale — a “live” feed that's quietly 90 seconds old. `StreamReader` drops old frames for a live source instead, keeping latency bounded. `core/ingest.py`
-
- 03 **YOLO11n finds people, ByteTrack keeps their identity.** One model, person class only (`classes=[0]`), `persist=True` so this frame is read as a continuation of the last one, not a fresh scene. One tracker instance per camera — sharing one across cameras would interleave four unrelated scenes into a single association problem. `core/person.py`
-
- 04 **YuNet looks for a face — only inside a person's own box.** Cascaded, not full-frame: searching a handful of small head crops instead of 2 million pixels, and it structurally cannot hallucinate a face onto foliage or a window reflection the way a full-frame detector can. Runs coarser than person detection (- - face - every) — one good sighting per track is enough. `core/face.py`
-
- 05 **Every box becomes the same wire shape.** `normalise()` is the one place a pixel box turns into `{track_ref, class, confidence, bbox}`, called once for each person and once for each face. A face's `track_ref` is derived from its owning person's (`<run>:<id>:face`) — a face has no ByteTrack identity of its own, so it doesn't get to pretend it can cross a zone independently. `core/ingress_client.py`
-
- 06 **The hot path goes out first.** `BoxChannel.publish()` pushes the frame's boxes to every browser watching that camera, over one shared websocket multiplexed by camera id. Sent even when empty — an empty list is how a tile learns a person *left*; skipping it would freeze the last box on screen and read as a dead detector. `core/box_channel.py`
-

- 07 **The cold path goes out on a background thread.** IngressClient POSTs the same detections to the backend. A blocked socket must never stall detection — a failed connect measured ~2s on this hardware despite a 1s timeout, and a queue of size 1 means only the newest frame is ever in flight. `core/ingress_client.py`
-
- 08 **The backend validates, then judges.** `/hooks/ingress/detections` is the one door — the same one the built-in simulator posts through. Past that, the fence decides: which side of the zone line is this, did it just cross, and has it held long enough (`confirm_seconds`) to reject a single frame's flicker. `backend/src/l4/hooks.ts`, `backend/src/l2/fence.ts`
-
- 09 **A real crossing becomes an event, then an incident.** Append-only, monotonic sequence number. Related events inside a time window group into one incident — an operator's unit of work is “someone crossed the fence,” not forty individual detections. `backend/src/l3/events.ts`
-
- 10 **The console shows three independent things at once.** Live video (WHEP), live boxes + a short client-side movement trail (WS), and the durable incident queue (SSE) — three channels that can each fail without taking the others down. `frontend/src/lib/{whep,boxes,stream}.ts`
-

03 Two channels, same detections, nothing in common but the source

Muxing these into one stream would mean a dropped websocket frame silently loses a durable record. Kept apart, the worst a slow detector can do is draw a late box — never lose one.

HOT Box channel

What a human is watching, right now.
Worthless a second after nobody is looking at the tile.

<code>carries</code>	raw boxes, every detector call
<code>transport</code>	WebSocket, one socket for the whole console
<code>durability</code>	none — never stored, never replayed
<code>file</code>	core/box_channel.py
<code>consumer</code>	lib/boxes.ts → camera-feed.tsx canvas overlay

COLD Ingress → fence → event log

What the fence decided about a detection. The only thing that has to survive a restart.

<code>carries</code>	same detections, HTTP POST
<code>transport</code>	background thread, one door: /hooks/ingress/detections
<code>durability</code>	append-only SQLite, hash-chained audit trail
<code>file</code>	core/ingress_client.py → backend/src/l2/fence.ts
<code>consumer</code>	lib/stream.ts (SSE) → Incidents, Event log

04 The wire contract

What actually crosses the process boundary — this is what to check first when a detection isn't showing up somewhere it should.

POST /hooks/ingress/detections COLD

```
{
  "camera_id": "cam_fence_north",
  "occurred_at": "2026-09-10T22:56:01.000Z",
  "capture_mono": 116153.671,           // monotonic; wall clock can step under NTP
  "simulated": false,                  // set once, by the adapter – never per
  detection
  "detections": [
    { "track_ref": "7ca8:6",           // <run_id>:<ByteTrack id>
      "class": "person",
      "confidence": 0.886,
      "bbox": [0.481, 0.135, 0.093, 0.525] }, // [x,y,w,h] normalised 0..1
    { "track_ref": "7ca8:6:face", // derived from the owning person's ref
      "class": "face",
      "confidence": 0.71,
      "bbox": [0.495, 0.140, 0.021, 0.031] }
  ]
}
```

Box channel message HOT

```
{ "t": "boxes", "camera_id": "cam_fence_north", "capture_mono": 116153.671,
  "boxes": [ /* same shape as "detections" above, minus occurred_at/simulated */ ] }
```

run_id prefixes every track_ref because ByteTrack restarts its counter from 1 every process start. The backend's `UNIQUE(camera_id, track_ref)` constraint means a bare id would let a brand-new person silently inherit a dead track's pending zone crossing. Scoping the ref to the run makes that impossible by construction, not by luck.

05 File-by-file reference

Grouped by which of the four processes actually runs the file.

Vision service – Python, `ibvap/`

FILE	ROLE
people_run.py	Single-source debug CLI. One video in, an overlay window out (--show), optionally posting to the backend. For looking at the detector itself, not the real shape.
people_service.py	The real multi-camera process. Reads media/cameras.yml + .env, one CameraWorker thread per camera, drives both channels. This is what a deployed post actually runs.
core/ingest.py	Shared. RTSP/file/webcam capture on its own thread — latest-frame-wins for a live source, zero-loss for a file. Source-aware on purpose; do not simplify to one behaviour.
core/person.py	YOLO11n + ByteTrack, person class only. PersonTracker.update(frame) → list of {track_id, bbox, conf}.
core/face.py	YuNet, cascaded inside each tracked person's head region. detect_for_persons() → list of {track_id, face_bbox, score}, keyed to the owning person.
core/ingress_client.py	Shared with the vehicle domain. normalise() is the one function that turns a pixel box into the wire shape — class_ is a required argument on purpose, so a domain can never silently mislabel the other's detections. IngressClient runs the POST on a background thread.
core/box_channel.py	Shared. The websocket server for the hot path — one asyncio loop, one socket, multiplexed by camera id.
core/settings.py	Shared. Reads .env (where things run) and media/cameras.yml (what cameras exist) — deliberately not merged, so every machine can read the same manifest with a different IBVAP_WORKER_CAMERAS.
data/face_detection_yunet_2023mar.onnx	YuNet weights, ~230 KB. Gitignored — must be fetched with GitHub's “Download raw file,” not a plain save (which yields a 131-byte LFS pointer).

Edge node — Bun + SQLite, backend/src/

FILE	ROLE
14/hooks.ts	The one door. <code>parseDetectionFrame</code> validates the POST body; <code>ingestDetections</code> hands it to the fence. The built-in simulator posts through this exact same function.
12/fence.ts	Per-track, per-zone judgement in RAM: which side of the line, did it cross, has the crossing held for <code>confirm_seconds</code> . A single frame's flicker is rejected here, not shouted.
13/events.ts	Writes the append-only event row and groups related events into an incident — an operator's actual unit of work.
14/bus.ts	Live push over Server-Sent Events at <code>/api/stream</code> — how a new incident reaches the console without polling.
db/schema.sql	<code>tracked_thing</code> , <code>event</code> , <code>zone/zone_camera/zone_target</code> — event is append-only, enforced by a trigger, not a promise.

Console — React, frontend/src/

FILE	ROLE
lib/whep.ts	WebRTC/WHEP client — the only way a browser can play what MediaMTX serves (browsers cannot play RTSP directly).
lib/boxes.ts	One websocket for the whole console, subscribed to the box channel, multiplexed by camera id client-side.
lib/stream.ts	One EventSource for the whole console, subscribed to the backend's SSE bus.
components/ibvap/camera-feed.tsx	Composes video + box overlay + a short client-side movement trail on a canvas, drawn from an animation frame — never re-renders React per box.
features/cameras/page.tsx	One camera's full record: live tile, its zone(s) on the map, and every incident it has produced.
features/live/screen.tsx	The camera grid — secondary by design; the incident queue is where the actual work happens.
features/incidents/screen.tsx	The operator's queue: ranked, worst first, keyboard-driven (ack / escalate / dismiss).

Media hub

FILE	ROLE
media/cameras.yml	The one place a camera is declared — id, label, source (file/RTSP/webcam), whether detection runs on it.
media/configure.py	cameras.yml + .env → mediamtx.yml. Re-run after editing either.
media/fetch.py	Resolves footage — downloads, trims, re-encodes to WebRTC-safe H.264 baseline (strips B-frames, which WebRTC cannot carry).

06 Stated honestly

Limitations this system has right now, not discovered mid-demo.

TRACKING

No re-identification. ByteTrack carries no appearance model — a long occlusion produces a *new* track id. Short occlusions recover via its low-confidence association pass. Never claimed as persistent identity.

FACES

Resolution-bound, and detection only. A 60 px-tall person has roughly a 12 px face — no detector finds that at range. YuNet returns a box and five landmarks; it does not identify anyone, and the word “recognition” is deliberately never used for this file.

RTSP

Missing reference picture decode warnings are cosmetic. They appear from the H.264 loop-boundary discontinuity in a looping demo clip and don't suppress detection — confirmed by running the same detector directly against a captured frame from the exact same stream.

THROUGHPUT

One CPU-only box, four workers, shared cores. No throughput figure is quoted here because none is claimed without being measured on the machine that ran it — `people_service.py` prints per-camera fps and detector ms/call at exit, and those are the only numbers worth repeating.

ibvap/people_service.py · 2 shared threads + 3 per camera · CPU-only — no CUDA path ·
reference document, generated 2026-09-10